Shade Walk

Solar-aware pedestrian routing · Project documentation, version 15 · Chris Kim

What it is	A mobile-first web app that plans walking routes with more shade (Shade mode) or less forecast rain (Rain mode), and compares them with the normal direct route.
The problem	Map apps give you the shortest walk, not the most comfortable one. In Brisbane's heat and summer storms, a slightly different street can mean much less sun or rain.
How I built it	I built it in about two and a half weeks (13–30 September 2026) with Claude as my coding assistant. I set the product direction, tested each version, decided what to keep or revert, and wrote this decision log.
Technology	One HTML file (about 2,700 lines of HTML, CSS and JavaScript), no build step and no server. MapLibre GL for the 3D map, SunCalc for sun position, and seven free public data services (listed in section 6).
Status	Working prototype: route planning, both comfort modes, a best-time-to-walk forecast and turn-by-turn walking navigation.

1. What the app does

- **Two comfort modes.** Shade mode finds the route with the most building and tree shade. Rain mode finds the route with the least forecast rain along the walk.
- **Two clear choices.** Every search shows a comfort route (Max Shade or Dry Route) next to the Direct Route, with distance, time, and the shade or dry percentage for each. If the two are the same path, the app says so and shows one "Best route" card.
- **Live 3D shadows.** Buildings are shown in 3D and their shadows move as you drag the time slider, so you can see where the shade will be at any time of day.
- **Shade breakdown.** Each route card shows how many metres are shaded by buildings and by trees, plus a one-line reason such as "Mostly shaded by buildings".
- **Best time to walk.** The Trip Comfort card scores the route hour by hour for the rest of the day and suggests a better start time if waiting would give noticeably more shade.
- **Walking navigation.** Live GPS position, compass heading, next turn, remaining distance, automatic rerouting if you go off route, and a "shade ahead" reading for the next stretch.
- **Weather at a glance.** Current temperature, conditions and UV index for the starting point.
- **Night screen.** After sunset there is no shade to find, so the app shows a "Good night" screen with tomorrow's sunrise and options to plan tomorrow's walk or check the rain.

2. How it works

Finding routes

The app asks a public walking-route service (OSRM on OpenStreetMap) for the direct route and up to two alternatives. The first route appears on screen straight away; shade and rain analysis then update the cards a moment later, so nobody waits on a loading screen.

Calculating shade

- **Building shadows.** Each building's shadow is its footprint stretched away from the sun. The length is *building height ÷ tan(sun altitude)* and the direction is exactly opposite the sun's compass bearing. Heights come from OpenStreetMap; when a height is missing, the app estimates it from the number of floors or the building type.
- **Tree shadows.** Mapped trees (single trees and tree rows) cast a capsule-shaped shadow. Height and canopy size come from the map data where available, otherwise from typical sizes for the leaf type.

- **Route percentage.** The app checks a point every 12 metres along the route, testing three positions across the width of the footpath. A point counts as shaded if any building or tree shadow covers it, so overlapping shade is only counted once.
- **One source of truth.** The shadows drawn on the map and the shade percentage on the cards come from the same set of shadow shapes (see section 4).

Choosing the Max Shade route

Each alternative gets a score: more shade scores higher and a longer detour scores lower. An alternative only replaces the direct route if it gives at least 5 percentage points more shade (or 180 more shaded metres) and is no more than 35% longer. When better building or tree data arrives in the background, the choice is re-run automatically.

Rain mode

The app samples points along each route and requests the hourly forecast for them (rain chance, rain amount, showers and weather type). It works out when you will reach each point, so the forecast matches the time you are actually there. A drier route must cut the rain risk by at least 5 points and be no more than 35% longer. A live radar overlay shows where rain has recently fallen. Forecast and radar are kept separate on purpose: the forecast says what is expected when you walk; the radar says where rain has just been.

Search

Search is forgiving: typing “Toowong vil” finds Toowong Village before you finish. Results are biased toward your current area, requests wait until you pause typing and cancel when you keep typing, and your five most recent places are remembered on your device.

3. Key design decisions

Keep the 3D buildings

Why: A flat map makes it hard to see which building is creating the shade.

Result: People can see the street and the buildings casting shadows onto it.

Soft, readable shadows instead of realistic ones

Why: The shadows exist to communicate where the shade is, not to look cinematic. Darker, sharper, more realistic shadows made the map cluttered.

Result: Shadows are one flat, semi-transparent layer. Overlapping shadows never stack darker, and the 3D buildings sit on top of them.

Always show both routes

Why: Users should never wonder whether the app failed to find an alternative or the routes are genuinely the same.

Result: Two cards when the routes differ; one “Best route” card, clearly labelled, when they are the same.

Stay on real walking paths

Why: A shadier route is useless if it sends you through a building or private property.

Result: Routes come from a pedestrian routing service, and any route that runs along a path mapped as private or no-access gets a “May cross a private path” warning.

Show the route first, analyse after

Why: Waiting for every data source before showing anything made the app feel slow.

Result: The route appears immediately; shade, trees and access checks load in the background and update the cards.

Night is a separate, always-dark screen

Why: At night there is no shade to calculate, and a normal map with “0% shade” would be confusing.

Result: A dedicated Good Night screen based on the real clock. Dragging the slider can never trigger it, because in Shade mode the slider only covers daylight hours.

Map-first phone layout

Why: Shrinking the desktop sidebar onto a phone covered most of the map.

Result: On phones, the search panel collapses to a one-line trip summary once a route is found, and route cards sit in a small swipeable shelf at the bottom.

4. Problems I solved along the way

Problem	Cause	Fix
Map showed shadows but the card said “0% Shade”	The map drawing and the percentage used different data paths (rendered layers, downloaded buildings, separate fallbacks) that were not ready at the same time.	Rebuilt the engine around one set of shadow shapes used for both drawing and scoring (v15). The percentage now always matches what you see.
Patchy shadows near the user	The map returns cached building tiles in no useful order, and a fixed cap sometimes dropped nearby buildings while keeping distant ones.	Keep only tiles near the view and route, most detailed first, and remove duplicates of the same building.
Shadows lagged behind the time slider	Every slider movement queued another full redraw.	Only the latest state is drawn, and while dragging the app draws a simpler version, then full detail on release.
App used about 50% CPU while idle	Each redraw triggered another “map idle” event, which triggered another redraw.	Only redraw when new building data actually arrives.
Shade % changed when you panned the map	Tree data was reloaded around the map centre, replacing the trees along the route.	Once a route exists, tree data is locked to the route corridor.
Free map-data servers often busy	The public Overpass servers rate-limit and time out.	Results are cached on the device for 24 hours, a second server is tried automatically, an older cached answer is used if all fail, and the card warns when data is limited.

Experiments I reverted

- **Heavy 3D shadow rendering:** shadows became dark, blocky and looked like extra buildings. I kept the useful part (moving, sun-accurate shadows) and dropped the visual weight.
- **Polygon-only shadow replacement:** produced awkward shadow blocks and removed the soft look people found easiest to read.
- **Changed shadow start-up state:** shadows looked different when the map first opened and only corrected after moving the slider. I restored the v15 behaviour.

5. Rules for future changes

- The v15 shadow appearance is the approved visual baseline. Solve other problems underneath it without changing the look.
- Keep one sun-direction conversion: SunCalc 1.9.0 measures azimuth from south, so it is converted to a compass bearing once, and shadows always point 180° away from the sun.

- Never “fix” a shade percentage by changing the displayed number. It must come from the real route and shadow calculation.
- Comfort routes must stay on walkable paths.
- Always show the comfort route and the direct route, and say clearly when they are the same.
- Change Shade, Rain, Routing, Navigation and the interface independently where possible, so fixing one does not break another.

6. Data sources and libraries

Service or library	Used for
MapLibre GL 5.20 + OpenFreeMap	The 3D map, base map tiles and building footprints with heights
SunCalc 1.9	Sun position, sunrise and sunset for any place, date and time
OSRM foot routing (routing.openstreetmap.de)	Walking routes, alternatives and turn-by-turn steps
Overpass API (OpenStreetMap)	Buildings and trees along the route, and paths mapped as private or no-access
Nominatim and Photon	Place search and the user’s local area for better search results
Open-Meteo	Current temperature, conditions and UV; hourly rain forecast along the route
RainViewer	Recent rain radar overlay

7. Known limitations

- Accuracy depends on OpenStreetMap. Missing buildings or trees mean missing shade, and many building heights are estimates.
- The model does not include awnings, verandas, bus shelters, terrain or cloud cover.
- Free public services can be slow or busy. The app degrades gracefully and warns the user, but results may be under-counted at busy times.
- The private-path check warns but does not yet steer the route away.
- Navigation is a prototype and has not been tested on long walks.
- The code is one large file with several layers of style fixes from successive versions.

8. Next steps

- **Test with real users** on common Brisbane walks and compare the shade % with what people actually experience.
- **Split the single file** into separate map, shade, rain, routing and interface modules, and merge the layered style fixes into one stylesheet.
- **Add automated tests** for the shade engine using fixed buildings and sun positions with known answers.
- **Route around private paths** instead of only warning about them.